NAZAR
How It Works

A plain-language guide to every page of the NAZAR satellite tracker — what each part does, the logic behind it, where the data comes from, and which bits were hard and which were easy. **No programming knowledge required.**

Built by HD Gala at the Takshashila Institution. Everything runs in your web browser — there is no server, no account, and no tracking.

The one idea that makes it all work

Every satellite the world tracks is published by NORAD (the US space-surveillance network) as a tiny text record called a **TLE** — a "Two-Line Element set". A TLE is two lines of numbers that describe an orbit: its shape, tilt, and where the satellite was at one exact moment.

From those two lines, a well-known piece of math called **SGP4** can calculate where the satellite is *right now*, or at any time you ask for. NAZAR does that math **inside your browser**, thousands of times a second, for the whole catalogue. That is the whole trick: download small text files, run the standard orbit math locally, and draw the results on globes and maps.

> **Why this matters**
> Because the heavy lifting happens on your own device, NAZAR needs no back-end server, no database, and no API key. It is just a set of web pages and a few data files hosted for free on GitHub Pages. That also means it keeps working offline-ish: once the data is cached, the visuals still run.

Where the data comes from

Two feeds from **CelesTrak** (a respected public clearing-house for orbital data):

- **TLEs** — the orbit records for every active satellite, fetched from CelesTrak's `gp.php` endpoint.
- **SATCAT** — the satellite catalogue, which adds the metadata TLEs lack: who owns each object, its launch date and site, its country, and whether it is a payload or debris.

CelesTrak politely rate-limits repeat callers, so NAZAR is defensive about it. A scheduled robot (a **GitHub Action**) re-downloads the whole catalogue **every 6 hours** and commits the fresh snapshot into the site itself. So the pages have a three-stage safety net: try the live feed, fall back to your browser's recent cache, and finally fall back to the bundled 6-hour-old snapshot. You almost never see a failure.

> **The hard part here**
> Rate limits. A brand-new visitor hitting the live feed can get throttled, so a lot of care went into fast time-outs, caching in your browser, and always shipping a good-enough snapshot with the site. The easy part: the data itself is just plain text files.

The shared engine

Every page loads one small shared file that does the orbital math so the rest of the site doesn't have to repeat it. It knows how to: fetch the TLEs (with all the fallbacks above), turn each TLE into a ready-to-run orbit object, and **propagate** — i.e. ask "where is this satellite at this instant?" and get back a latitude, longitude and altitude. Everything visual on the site is built on top of that one answer.

The pages, one by one

NAZAR is a set of linked pages, each a different lens on the same live catalogue. Here is what each one does.

1 · The Main Page — the live globe

How it works. A realistic 3-D Earth (the globe.gl library) with satellites shown as dots at their real sub-points. You pick an **observer city** in India (or all ten, or a China-only mask) and the page counts how many satellites are currently **above that city's horizon**, split into Chinese and non-Chinese. Click any dot to trace the ground it covered over the last 30 minutes and where it is heading. A **See all satellites** pop-up opens a second globe with *every* above-horizon object at once, and a ticker scrolls the latest Chinese launch news.

Where the data comes from. Live TLEs for the whole active catalogue, propagated in your browser; the Chinese-vs-rest split comes from the SATCAT owner field.

The hard part vs the easy part. Hard: deciding which satellites are 'visible' from a city means look-angle geometry (azimuth/elevation) for thousands of objects several times a second. Easy: drawing a dot at a latitude/longitude once you have the position.

4 · Orbits & Ground Tracks — the 2-D view

How it works. Search any satellite and its **ground track** is drawn on a flat world map: the past 24 h and next 24 h as dotted lines, with an arrow every 30 minutes showing direction and the place it is over on hover. A **Time / Revolution** toggle switches to an animation where a speed slider (1× up to 5000×) flies the satellite along its orbit, tracing a golden path with a live projected UTC and IST clock. Two small 3-D globes to the side stay perfectly in sync: one keeps the satellite **centred**; the other spins the Earth at its **true rotation rate** so you see the orbit's real shape — a near-circle for low orbits, a long ellipse for HEO orbits — with the dot blinking whenever it passes behind the planet.

Where the data comes from. One satellite you choose; its full orbit is computed from its own TLE. Place-names on hover come from an offline country-outline dataset.

The hard part vs the easy part. Hard: the second globe. To make the satellite trace a fixed orbit while the Earth spins under it, the camera is quietly orbited at the Earth's true rotation angle — a little trick from inertial mechanics. Splitting the track where it crosses the date-line, and fading the dot when it's occluded by the globe, were fiddly too. Easy: a flat map is just longitude→x and latitude→y, a straight-line mapping.

7 · 3-D Visualiser — the God's-eye view

How it works. All ~16,000 satellites at once, each at its real height above the Earth, colour-coded by orbit class (low, medium, geostationary, elliptical). Hover any dot for its altitude, speed and period. Click one — or type its name in the search box at the bottom — to **spotlight** it: the rest of the swarm dims away and the chosen satellite's complete orbit is drawn as a ring. Click empty space to release.

Where the data comes from. The entire active catalogue, propagated in browser and placed at true altitude.

The hard part vs the easy part. Hard: drawing sixteen thousand moving spheres without the browser choking. The trick is to render them all as a single 'instanced' object — one instruction to the graphics card instead of sixteen thousand. Working out which tiny dot your cursor is over is also non-trivial. Easy: the colour-by-class rule is a simple altitude threshold.

8 · SATs-by-OPs — coloured by operator

How it works. The same God's-eye swarm, but colour-coded by **who operates each satellite** — Maxar, Capella, ICEYE, all of Starlink, the whole BeiDou constellation, and about 37 categories in all, each with its own on/off toggle. Hover an operator's name and its fleet pulses on the globe; click the name to open a **dossier** explaining who they are and what they fly.

Where the data comes from. Same catalogue; each satellite is matched to an operator or constellation by name pattern.

The hard part vs the easy part. Hard: there is no clean 'operator' field, so satellites are sorted into families by matching their names — a big hand-tuned list of patterns. Easy: once sorted, toggling a colour on or off is instant.

2 · SAT-STATS — the numbers

How it works. Every tracked satellite sorted by **country of origin** — pick India and see how many of our vehicles are up there. It keeps a **cumulative** count in your browser, so the numbers only ever grow as new objects appear, and a **Previously tracked** pop-up lets you search everything ever seen with a country filter. The charts underneath (drawn with Chart.js) are eye-opening — some single launch sites have flown more rockets than India has satellites in orbit.

Where the data comes from. SATCAT metadata (country, launch site, launch date) joined with the live catalogue; a running total is kept in your own browser.

The hard part vs the easy part. Hard: making the counts 'cumulative' means remembering across visits which objects you have seen — that is what your browser's local storage is for. Easy: once you have the tallies, a bar chart is a library call.

9 · Play NAZAR — the orbit tracer with a soundtrack

How it works. A 3-D view that draws each satellite's **orbital track** as a continuous loop in space. Press ■ **Play NAZAR** and the screen goes full-bleed while the globe gyrates in time with a soundtrack. It began as a happy accident — early glitchy orbit lines looked too good to delete — so it got music. Entirely not-useful, and the most fun to build.

Where the data comes from. Each satellite's orbit is sampled across one full period from its TLE.

The hard part vs the easy part. Hard: syncing camera moves to the beat. Easy: one line per orbit is straightforward once the orbit math is in hand.

3 · TLE Repository (inside SAT-STATS)

How it works. A technical corner that shows the **raw Two-Line Elements** themselves, plus a layman's decoder that explains what every character of a TLE means. Fun to try to read one.

Where the data comes from. The raw TLE text for every tracked vehicle.

The hard part vs the easy part. Hard: explaining checksums and epoch formats simply. Easy: displaying text you already have.

5 · China-Sat-Repo, Compendium & Field Guide

How it works. A live, filterable **table of every active Chinese payload** that untangles the confusing naming (Yaogan vs Gaofen vs Shiyan…). It links out to a deeper **Compendium** of every PRC programme and a **Chinese Satellite Series field guide** — reference documents for anyone trying to make sense of China's fleet.

Where the data comes from. SATCAT filtered to Chinese (PRC-owned) payloads, joined with live positions; the reference pages are hand-written catalogues.

The hard part vs the easy part. Hard: the research — mapping opaque programme names to what each series actually does. Easy: presenting it as a searchable table.

6 · Game of Cones — the geometry playground

How it works. Two satellite-geometry puzzles. **Land Cone**: stand at a latitude/longitude, define an upward cone of a chosen angle, and see every satellite inside it — what sits at which angles when you look up. **Sat Cone**: the inverse — pick a satellite, set its sensor angle, and see the patch of ground it can 'see', drawn as a footprint on the Earth.

Where the data comes from. The live catalogue plus a bit of trigonometry you control with sliders.

The hard part vs the easy part. Hard: the cone-to-sphere geometry (where a cone intersects the round Earth). Easy: sliders wired to redraw.

What is stored about you

Nothing personal — no account, no email, no analytics of you. The only things NAZAR saves are **satellite data** and a few of your **preferences**, and they live entirely in your own browser's local storage for speed:

- the most recent catalogue snapshot, so pages load instantly on a repeat visit;
- SAT-STATS' cumulative tallies, so its counts keep growing across visits;
- small UI choices like which globe you last looked at.

None of it leaves your device. Clearing your browser data resets all of it with no loss to anyone else.

How it's put together and shipped

NAZAR is a **static website** — a folder of HTML, CSS and JavaScript files with no build step and no server. It is hosted for free on **GitHub Pages**. Publishing a change is just committing to the repository; the site rebuilds automatically in under a minute. The 6-hourly data robot commits fresh snapshots the same way, so the catalogue stays current on its own.

> **The honest summary**
> The 'hard' of NAZAR is mostly performance and defensiveness — running real orbital math for sixteen thousand objects smoothly, and never letting a rate-limited feed break the page. The 'easy' is that, once you have a satellite's position, showing it — a dot on a globe, a line on a map — is the fun, fast part. Almost everything you see is that one position, drawn many different ways.